

FRACMATH: A vectorized MATLAB framework for continuum damage mechanics with crack band regularization

Jaykumar Mavani^a, Madura Pathirage^{a,*}

^a*Gerald May Department of Civil, Construction, and Environmental Engineering, University of New Mexico, Albuquerque, NM, USA*

Abstract

FRACMATH is an inspectable, vectorized MATLAB implementation of isotropic damage with direction-dependent crack-band regularization. Its code links projected width, constitutive history, sparse assembly and solution. Material-point tests verify strain mapping, irreversible damage and fracture-energy calibration. Mesh/increment studies compare regularized and constant-width softening curves. Ninety observations of thirty matched computing configurations assess MATLAB CPU, hybrid GPU and Abaqus SMP configurations. The hybrid GPU shows no speed advantage on the tested workstation. A geometry-matched three-dimensional pure-tension comparison supplies experimental error measures; mixed-mode and torsion images illustrate workflows. Source, histories, diagnostics and reproduction scripts accompany the paper.

Keywords: continuum damage mechanics, crack-band regularization, MATLAB, hybrid GPU, Abaqus UMAT, quasibrittle fracture

Required Metadata

Current code version

Nr.	Code metadata description	Value
C1	Current code version	v1.1.1, commit f4d208e
C2	Permanent link to code/repository used for this code version	Core software archive: immutable GitHub commit ; timing and figure companions: immutable companion commit ; version archive DOI .
C3	Legal code license	MIT License
C4	Code versioning system used	git
C5	Software code languages, tools and services used	MATLAB; Fortran UMAT; Abaqus/Standard; Python analysis scripts
C6	Compilation requirements, operating environments and dependencies	MATLAB R2024b; Abaqus/Standard 2024 with Intel Fortran; Python 3 with NumPy, SciPy, Matplotlib, Pillow, and psutil; Parallel Computing Toolbox and a compatible CUDA GPU for the optional hybrid backend. Verified on Windows 11.
C7	If available, link to developer documentation/manual	Beginner guide ; implementation walkthrough and reproduction guides
C8	Support email for questions	mpathirage@unm.edu

Table 1: Code metadata.

*Corresponding author

Email address: mpathirage@unm.edu (Madura Pathirage)

1. Motivation and significance

FRACMATH is intended for researchers and students who need to inspect the numerical path from a strain field to a damage field and a global response. The constitutive law and crack-band principle are established methods [1, 2, 3]. The code shows the assumptions, update order, regularization length and numerical limits so that users can check and change them. The package therefore includes element-level routines, a 2D bending case, two 3D examples, plotting scripts, and an Abaqus UMAT for independent comparison. The controlled study examines how element-size compensation affects structural response and dissipation, and how increment size and sparse linear algebra limit the numerical workflow.

1.1. Related software and scope

General-purpose finite element systems such as Abaqus [4] and OOFEM [5] provide broader element and analysis capabilities. CALFEM [6] provides an accessible MATLAB finite element teaching environment. FRACMATH offers a narrower, code-readable example of direction-dependent local crack-band damage in 2D and 3D. It is designed for teaching, replication, and testing method changes.

2. Software description

2.1. Software architecture

The package follows a visible sequence: input settings and mesh files; precomputed element operators; displacement increments and constitutive history; sparse equilibrium solution; saved numerical states; and figure reconstruction. The 2D entry `start_here.m` exposes material, increment and backend settings and calls the benchmark solver. Mesh inputs contain coordinates, connectivity and support/loading sets. Outputs contain load-CMOD histories, damage snapshots, residuals and component timings. The beginner guide describes units and array dimensions; the implementation walkthrough maps each output to its producing function. Separate 3D drivers retain their documented equilibrium algorithms. The optional UMAT and hybrid GPU paths share the 2D constitutive definitions while using different execution and equilibrium procedures.

2.2. Software functionalities

A scalar damage variable $\omega \in [0, 1]$ scales the elastic stress:

$$\boldsymbol{\sigma} = (1 - \omega)\mathbb{C}_0 : \boldsymbol{\varepsilon} \quad (1)$$

The history variable retains the maximum equivalent strain:

$$\kappa_{n+1} = \max(\kappa_n, \tilde{\varepsilon}_{n+1}) \quad (2)$$

Damage irreversibility is imposed by

$$\omega_{n+1} = \max(\omega_n, \omega_{\text{trial}}) \quad (3)$$

This also applies when the projected width changes. The modified von Mises measure is

$$\tilde{\varepsilon} = \frac{k-1}{2k(1-2\nu)}I_1 + \frac{1}{2k}\sqrt{\left(\frac{k-1}{1-2\nu}\right)^2 I_1^2 + \frac{12k}{(1+\nu)^2}J_2}. \quad (4)$$

$$J_2 = \frac{(e_1 - e_2)^2 + (e_2 - e_3)^2 + (e_3 - e_1)^2}{6}, \quad (5)$$

where $k = f_c/f_t$. For the 2D plane-stress element,

$$e_3 = -\nu(e_1 + e_2)/(1 - \nu). \quad (6)$$

Both the MATLAB and UMAT implementations use Eq. (4).

The damage law is

$$\omega(\kappa) = 1 - \frac{\kappa_0}{\kappa} \exp \left[-\frac{\kappa - \kappa_0}{\varepsilon_f - \kappa_0} \right], \quad (7)$$

$$\kappa_0 = f_t/E. \quad (8)$$

$$\varepsilon_f = \frac{\kappa_0}{2} + \frac{G_F}{h f_t}. \quad (9)$$

The Oliver projected width for a three-node triangle is

$$h(\mathbf{n}) = \frac{2}{\sum_{a=1}^3 |\nabla N_a \cdot \mathbf{n}|}, \quad (10)$$

with $\mathbf{n}$ along maximum principal strain. The 3D tetrahedral implementation uses the analogous projected width. Its exponential law uses

$$\beta = f_t h / G_F. \quad (11)$$

$$\omega = 1 - (\kappa_0/\kappa) \exp[-\beta(\kappa - \kappa_0)]. \quad (12)$$

Damage is capped at 0.999999. Here G_F calibrates post-peak work; Eq. (7) calibrates total uniaxial work in 2D. These are local regularization formulas; the material-point energy check below does not by itself prove structural mesh objectivity. The pure-tension 3D implementation retains both κ and ω ; accepted increments commit their nondecreasing histories, while rejected trials leave the committed state unchanged.

2.3. Numerical implementation

The 2D driver `solver_main_3pb.m` reads the mesh and support/loading sets, precomputes elastic element matrices and shape-function gradients, and advances prescribed displacement. At each increment it (1) assembles the secant stiffness from the previous damage field, (2) factors its free block with `decomposition`, (3) solves equilibrium for the frozen damage state, (4) obtains element strains with `pagetimes`, updates h , κ , and ω in `damage_update`, and (5) reassembles the damaged stiffness for reaction forces and the next increment. The factorization helper first attempts Cholesky and falls back to LU if that attempt fails. It is reused only within a displacement increment. There is no outer equilibrium iteration after the damage update, so this is a sequential secant scheme rather than a fully converged Newton solve. The post-update free-DOF residual is exported for each increment.

For example, the projected-width calculation acts on each triangle's three gradient pairs. With `gradN`, `nx` and `ny` denoting those gradients and principal-direction components, its MATLAB form is

```
projected = gradN(:,1:2:end).*nx + gradN(:,2:2:end).*ny;
h = max(2./max(sum(abs(projected),2),1e-14),1e-12);
```

This width enters Eq. (7); `kappa` and `omega` retain the irreversible element histories.

The optional Abaqus path uses `run_3pb_abaqus_OLIVER_T3_FAST.py` to write a table of T3 shape-function gradients. The Fortran `cdm_umat_2d_OLIVER_T3_FAST.for` reads that table to compute Eq. (10); it does not use `CELENT` when the table is present. The UMAT supplies the degraded elastic secant matrix as `DDSDDE`, rather than a consistent derivative of the evolving damage law. Its approximate tangent requires reporting equilibrium iteration counts with timings. `UEXTERNALDB` initializes the gradient table before SMP workers start; material calls read its saved arrays, and missing gradients terminate the job.

The optional 2D hybrid GPU backend keeps strain operators and reference element matrices on the device. A fused `gpuArray.arrayfun` kernel evaluates projected width, equivalent strain, and irreversible history. Results and stiffness values return to CPU sparse assembly and factorization. Timed operations include transfers and synchronization.

2.4. Verification and reproducibility

Verification uses executed MATLAB/Fortran routines, archived solver outputs and explicitly identified experimental digitization. Figures are deterministic plots of these data. The beginner entry reproduces benchmark numerical arrays exactly; rebuilding the seven generated figure assets gives identical pixels, and the two supplied geometry illustrations retain their source bytes. These checks establish reproduction of the archived outputs; illustrative 3D fields retain the validation limits reported below.

One-element tests check equivalent strain, unloading and reloading. A separate tension test checks the energy per crack area:

$$G_{\text{calc}} = h \int_0^\infty \sigma \, d\varepsilon. \quad (13)$$

Here h is the element’s projected crack-band width. The target is $G_F = 0.090$ N/mm. The Oliver model adjusts the softening curve using h , so the calculated energy stays close to this target at every tested width. For widths of 0.5, 1, 2 and 4 mm, its calculated energy is 0.090 N/mm to three decimal places. The constant-width control instead uses $h_{\text{ref}} = 1.25$ mm to set the same stress–strain curve for all elements. Its calculated energy therefore changes with the actual width h ; this is the expected control response. Over the same widths, its energy increases from 0.036 to 0.288 N/mm. The full material-point values are provided in the code archive. A driver linked to the unchanged UMAT reproduces the MATLAB energy values. Sixteen additional width/direction combinations pass, with maximum relative energy error below 0.00018%. These local checks do not establish structural orientation independence. An 824-case audit compares the actual UMAT with an independent tensor-eigensolve calculation and the MATLAB damage functions. The cases include 112 added unloading, reloading and rotating-strain states. Strain splits, multiaxial states, irreversible history, projected width, stress and the documented secant matrix pass declared absolute/relative tolerances of 2×10^{-10} . This does not establish a consistent damage tangent.

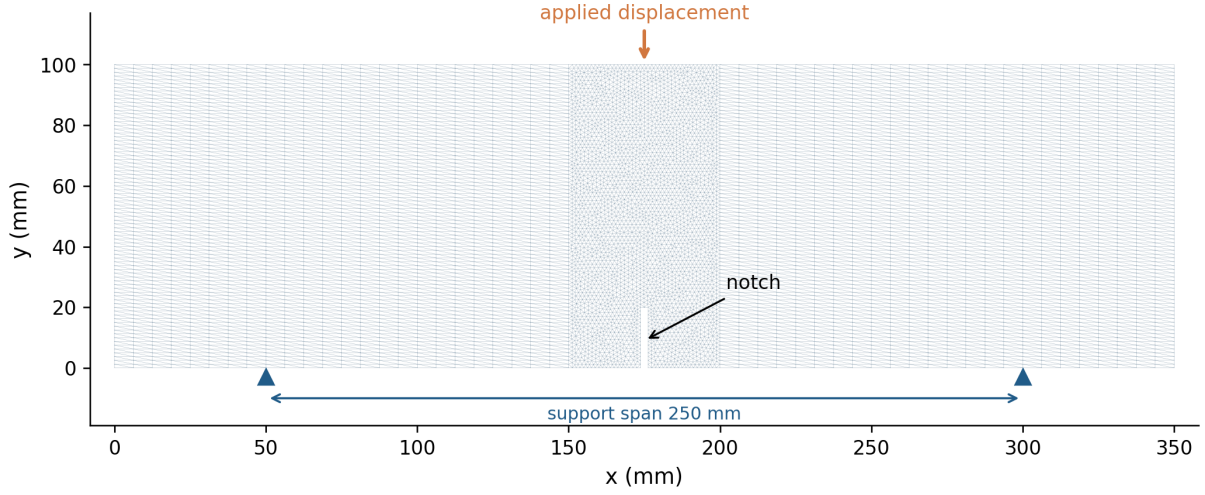
3. Illustrative examples

3.1. Benchmark 1: Notched 2D three-point bending

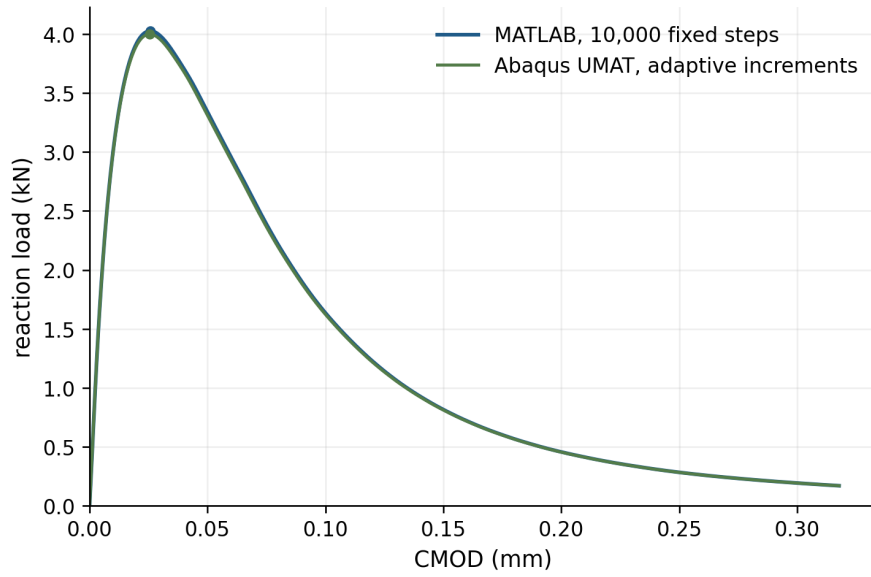
The Grégoire beam [7] has depth $D = 100$ mm, support span $S = 250$ mm, overall length 350 mm, thickness 50 mm, and a midspan notch of depth 20 mm and width 2.5 mm. Concrete parameters are $E = 37,000$ MPa, $\nu = 0.20$, $f_t = 3.5$ MPa, $f_c = 35$ MPa, and $G_F = 0.090$ N/mm. The 14,268 CPS3-element mesh has 7,319 nodes. The left support is pinned, the right is a roller, and a prescribed vertical displacement reaches -0.2 mm. CMOD is the mean horizontal displacement difference across the notch mouth. Figure 1 uses the mesh drawing: the 250 mm dimension denotes support span, while 350 mm is the overall length.

Figure 1 compares the 10,000-step MATLAB response with Abaqus. MATLAB peaks at 4.03 kN and CMOD 0.02579 mm; Abaqus peaks at 4.00 kN and 0.02540 mm, differences of 0.68% and 1.51%. A separate 1,000-step check peaks at 4.26 kN, demonstrating increment sensitivity. Peak post-update residuals are 0.016 and 0.002 for 1,000 and 10,000 steps. The overlay is a code-to-code check; increment-converged experimental validation is not established.

The completed Abaqus `.msg` reports 1,136 accepted increments, 4,817 equilibrium iterations/solver passes, 4,799 matrix decompositions, one cutback, and 45 acceptances under the alternate force tolerance. Its summed per-pass sparse-solver elapsed time is 100.2 s out of 832 s reported wall clock. The remaining wall time includes element/UMAT operations, assembly, convergence and output, which the `.msg` does not separate. Thus no attribution of the remainder to the UMAT or speed ranking between codes is supported. The MATLAB run used 10,000 fixed increments, while Abaqus used automatic increments, so the two wall-clock totals are not a controlled benchmark. The archived logs and timing parser reproduce these measurements. The controlled studies additionally retain per-step costs, residuals and iteration counts linked to the implementation walkthrough.



(a) Geometry, mesh, supports, and imposed displacement.



(b) MATLAB 10,000-step response and Abaqus adaptive-increment response.

Figure 1: 2D three-point bending case.

Figure 2 compares the severely damaged element bands at peak load and at the final saved state. The Abaqus field output is taken from the nearest stored frame, while the MATLAB panels use the state arrays.

The 10,000-step headless run used MATLAB R2024b, one computational thread, Windows 11, an Intel Core i9-13900 and 64 GB RAM. Tables 2–3 distinguish MATLAB assembly, factorization, damage and solves from Abaqus sparse-solver passes and unallocated elapsed time. Different algorithms, convergence quality and timing scopes qualify cross-program comparisons.

Table 2: Measured MATLAB 2D timing for the 10,000-step headless run. Other is total wall time minus the instrumented scopes.

Component	Time (s)	Share
Assembly	174.91	22.3%
Sparse factorization	519.54	66.1%
Damage update	17.95	2.3%
Free-block solve	71.52	9.1%
Other	1.78	0.2%
Total	785.70	100%

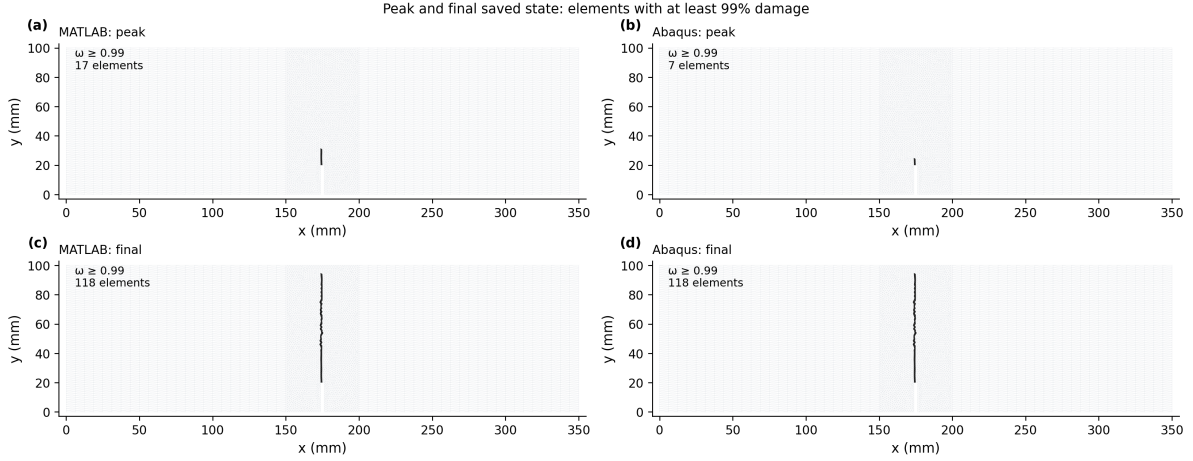


Figure 2: Damaged element bands on the full undeformed beam. The light grey background shows the CPS3 mesh. Dark elements have $\omega \geq 0.99$ in all panels. Left: MATLAB; right: Abaqus/UMAT. Top: peak load. Bottom: final saved state. The final selected band extends from the notch tip at $y = 20$ mm to about $y = 94.8$ mm in both programs. This is a severe-damage band, not complete separation: the stricter $\omega \geq 0.999999$ criterion reaches only $y = 55.4$ mm. Counts are printed in each panel. No smoothing or artificial crack extension is used. Abaqus uses stored field frames.

Table 3: Abaqus/Standard elapsed time extracted from the job’s `.msg`. The remaining time cannot be partitioned into UMAT, assembly, convergence, output, or overhead using this record.

Component	Time (s)	Share
Summed sparse-solver passes (4,817)	100.15	12.0%
Remaining wall time (unallocated)	731.85	88.0%
Total Abaqus wall clock	832.00	100%

Size, mesh and computing configurations

Thirty configurations, each measured three times, combine depths of 100/200 mm, three meshes, and MATLAB CPU1/CPU8/hybrid GPU or Abaqus CPU1/CPU8. All dimensions scale by two; final displacement is $-0.1/-0.2$ mm. Absolute global/notch seeds are 12.5/2.5, 6.25/1.25 and 4.6875/0.9375 mm; each computing comparison uses identical mesh files. The largest mesh has 100,104 elements and 101,088 DOFs. MATLAB uses 2,000 fixed increments; Abaqus uses maximum increments of 1/2,000, with SDV every 100 accepted increments. The two finest large-specimen Abaqus meshes use limits of 80 equilibrium iterations and 12 attempts, retaining default convergence tolerances; a failed default-control attempt is archived. Fresh processes run sequentially; the third observation reverses the computing-configuration order. MATLAB CPU8 limits numerical-library threads; the hybrid backend uses an NVIDIA RTX A2000 12GB in double precision.

All hardware-response checks pass: MATLAB load-history differences are below 1.76×10^{-11} of peak load and final-damage differences below 1.34×10^{-9} ; Abaqus pairs have identical curves at CSV precision and identical iteration counts. MATLAB CPU8 and hybrid GPU are slower than CPU1 in all six cases. CPU1 factorization occupies 67–76% of loop time. Abaqus SMP speed ratios range from 0.68 to 2.90; the largest mesh reaches 2.90. Abaqus CPU1 sparse-solver passes occupy 9.5–14.4% of analysis wall time; the remainder is unallocated. MATLAB/Abaqus peak differences are 1.87–4.24%. Table 4 reports median costs from 90 runs; complete phase, memory and residual records accompany the code. All saved MATLAB numerical arrays and Abaqus response CSVs are identical across repeats within each setting.

Separate small/coarse, single-CPU diagnostic runs reproduce all 2,001 baseline response rows and seven mesh/boundary hashes exactly. Instrumentation around the unchanged UMAT calculations records 18,579,456 calls and a raw elapsed sum of 2.996 s. The empty-clock-pair projection is 1.672 s; it is not subtracted because it does not measure the full instrumentation overhead. A separate native-only VTune profile estimates 2.873 s of CPU self time in four named assembly

routines. This excludes callees and unidentified work, so it does not measure full assembly wall time. These diagnostics use a different problem and timer scope from Table 3; they are separate from the 90 benchmark observations. The full reports, calibration, source files and response checks are available in the code archive.

A further serial rerun separates seven UMAT blocks and reproduces the same baseline exactly. Their raw elapsed sums total 11.899 s, while the projection of seven empty clock pairs is 10.404 s. Clock overhead and resolution therefore prevent reliable uninstrumented block percentages. The detailed block measurements are supplied as diagnostics in the archive rather than as a complete material-time breakdown.

A standalone benchmark links the unchanged UMAT and places high-resolution clocks outside batches of 14.24 million calls. Ten batches in two fresh processes give a median elapsed time of 1.148 s (range 1.129–1.241 s), including driver work. Empty-driver batches have a median of 0.128 s and are reported separately without subtraction. These cached material-point measurements reduce per-call clock intrusion; they are not structural Abaqus phase times and do not resolve individual-block costs.

Table 4: Size/mesh computing study: median time in seconds from three sequential observations per setting. MATLAB covers the load loop; Abaqus covers analysis and output. The data archive contains the observed ranges. GPU denotes the hybrid backend.

Size	Mesh	DOFs	MAT 1	MAT 8	MAT GPU	ABA 1	ABA 8
Small	coarse	3,772	31.3	33.8	38.5	158	234
Small	medium	14,686	131.2	144.0	146.1	832	403
Small	fine	25,534	232.3	251.8	255.0	1667	691
Large	coarse	14,686	127.6	140.0	147.0	819	398
Large	medium	56,954	598.9	639.6	626.4	4380	1586
Large	fine	101,088	1155.5	1206.0	1193.9	8548	2947

3.2. Benchmark 2: 3D Nooru-Mohamed Tests

The Nooru-Mohamed specimen is a square double edge notched concrete panel, $200 \times 200 \times 50$ mm. The illustrative mesh uses two 20 mm-deep, 5 mm-wide horizontal notches from opposite edges; the published specimen has 25 mm-deep notches [8]. The central ligament between the notch tips experiences a mixed mode stress state. The illustrative proportional-displacement case prescribes a vertical (mode I) displacement on the top edge and a proportional horizontal (shear) displacement on the side edges, with a shear to tension ratio $\gamma = 0.6$ and the bottom edge held fixed, ramped together under displacement control over 900 increments to a top displacement of 0.5 mm. The material is concrete with $E = 29,000$ MPa, $\nu = 0.20$, $f_t = 3.0$ MPa, $G_F = 0.110$ N/mm, and $k = f_c/f_t = 10$ (so $f_c = 30$ MPa).

The proportional-loading damage panels illustrate the workflow. The saved states do not all meet the equilibrium tolerance, so these panels are used only as a qualitative example. The option name **4c** selects proportional displacement control, not the published sequential experimental loading. A separate pure-tension comparison uses specimen 47-05 with two glued platens (Figure 3.16, printed page 45 of [8]). Its load–displacement curve is manually digitized; estimated reading uncertainty is ± 0.00102 mm and ± 164 N. This quantitative case uses the published 25 mm-deep, 5 mm-wide notches, with 35,917, 94,525 and 144,791 TET4 elements (21,828, 54,540 and 84,774 DOFs). Global/notch mesh seeds are 12/4, 9/3 and 6/2 mm. Material parameters above are retained without fitting. Bottom vertical displacements are fixed, the top platen has a common unknown vertical displacement, and minimal lateral pins remove rigid modes. The mean displacement of four interpolated 65 mm vertical gauges at $x = 30/170$ mm on both faces controls the load history. An augmented Newton system solves this constraint with nodal equilibrium; a numerical constitutive tangent differentiates the same damage and projected-width update. The relative equilibrium tolerance is 10^{-6} ; unconverged trials are rejected and their target intervals bisected without committing damage. The initial 600 target intervals to 0.2 mm produce 608, 606 and 601 accepted increments after eight, six and one rejected/bisected trials. Coarse, medium and fine peaks are 16.64,

16.40 and 16.53 kN, versus the digitized 19.85 kN: errors are -16.18% , -17.39% and -16.76% . The peak spread is 1.46% of their mean. On 1,001 uniform points over 0–0.2 mm, experimental curve RMS errors are 9.27%, 9.25% and 9.09% of experimental peak; medium–fine RMS difference is 0.35% of fine peak. Both peak and late softening tail are underpredicted. All accepted residuals meet 10^{-6} ; the maximum is 9.84×10^{-7} and maximum gauge error is 1.24×10^{-15} mm. The small mesh spread does not remove the experimental discrepancy or establish general mesh independence. Idealized platens, constitutive assumptions, digitization and untested increment convergence limit interpretation.

3.3. Benchmark 3: 3D Notched Beam Torsion

The torsion illustration uses a $400 \times 100 \times 100$ mm beam with one end fixed and the other prescribed a rigid twist to 0.003 rad over 140 increments. Material parameters are $E = 35,000$ MPa, $\nu = 0.20$, $f_t = 3$ MPa, $G_F = 0.080$ N/mm and $k = 10$. The TET4 mesh is refined near the notch front; damage updates use under-relaxation 0.35, and Cholesky solves reuse an AMD ordering.

The published Brokenshire test uses collar loading and measures applied load against clip-gauge CMOD [9, 10, 11]. The model has different end constraints and lacks a valid experimental CMOD channel. Figure 4 therefore illustrates nonplanar damage growth qualitatively; quantitative response validation and damage-path comparison are outside this example.

3.4. Controlled mesh sensitivity and computational scaling

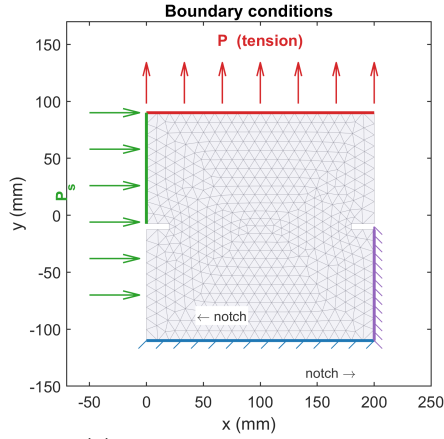
A controlled 2D mesh family contains 14,313, 25,042, and 56,216 CPS3 elements, corresponding to 14,686, 25,534, and 56,954 DOFs. The notched-beam geometry and material properties are unchanged. Additional partitions place the supports exactly at $x = 50$ and 300 mm and define a fixed 6.25 mm loading strip at the top centre; all nodes on that strip receive the same displacement. The 50 mm-wide central zone containing the notch is refined across the beam height. Its nominal seeds are 1.25, 0.9375, and 0.625 mm; the outer seeds are five times larger. This concentrates elements near the expected damage path while retaining a coarser outer region. These boundary sets and each complete Abaqus connectivity are exported directly to MATLAB; mesh-file hashes are checked after Abaqus rebuilds. This family is distinct from the single-mesh increment study. Both programs use one computational thread/CPU.

The main MATLAB runs impose -0.1 mm in 2,000 increments ($|\Delta u| = 5 \times 10^{-5}$ mm). The regularized runs update the Oliver projected width; the control runs hold the softening calibration width at $h_{\text{ref}} = 1.25$ mm on every mesh. This control uses one stress–strain softening curve that is identical on all meshes. It uses the exponential damage law in Eq. (7), with $h = 1.25$ mm for every element. Its softening rate is therefore independent of the actual element size. We call this the constant-width control; the Oliver runs instead calculate h from each element and the principal-strain direction. A second complete MATLAB family halves $|\Delta u|$ on all three meshes for both calibrations, using 4,000 increments. The Abaqus jobs use the same maximum displacement and increment cap, with automatic cutbacks available; their accepted counts are reported.

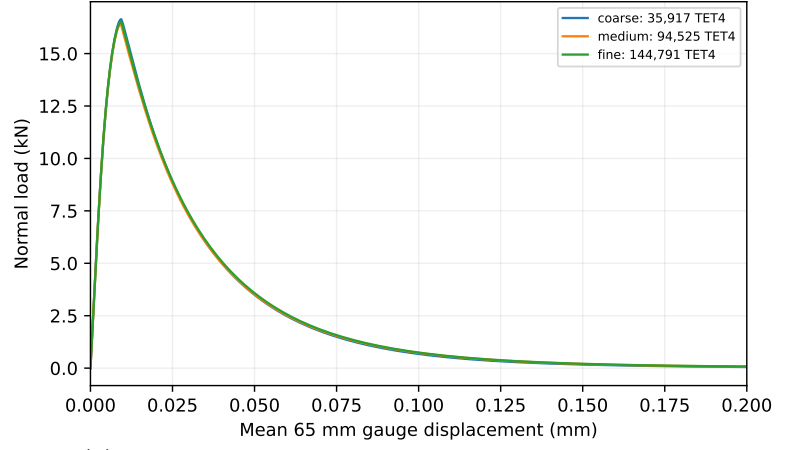
Damage dissipation is calculated by adding the undamaged elastic energy density times the damage increase and element volume over all increments. External work is integrated from reaction and imposed displacement. Stored energy uses the damaged elastic stiffness. Their balance is checked as a numerical diagnostic. Comparisons use CMOD 0.10 mm, so they measure partial structural dissipation rather than complete fracture energy.

Figure 5 compares the two softening calibrations. Coarse, medium and fine load–CMOD curves are overlaid separately for Oliver MATLAB, constant-width MATLAB and regularized Abaqus. The three MATLAB/Abaqus pairs are also plotted in the code archive; their full responses reveal peak and post-peak differences that timings cannot assess. Across the three main runs, peak-load spread, defined as $(\max - \min)/\text{mean}$, is 6.19% with Oliver regularization and 6.78% with the constant-width control. The corresponding dissipation spreads are 6.86% and 18.21% (Table 5). The comparison measures the effect of element-size compensation for this mesh family.

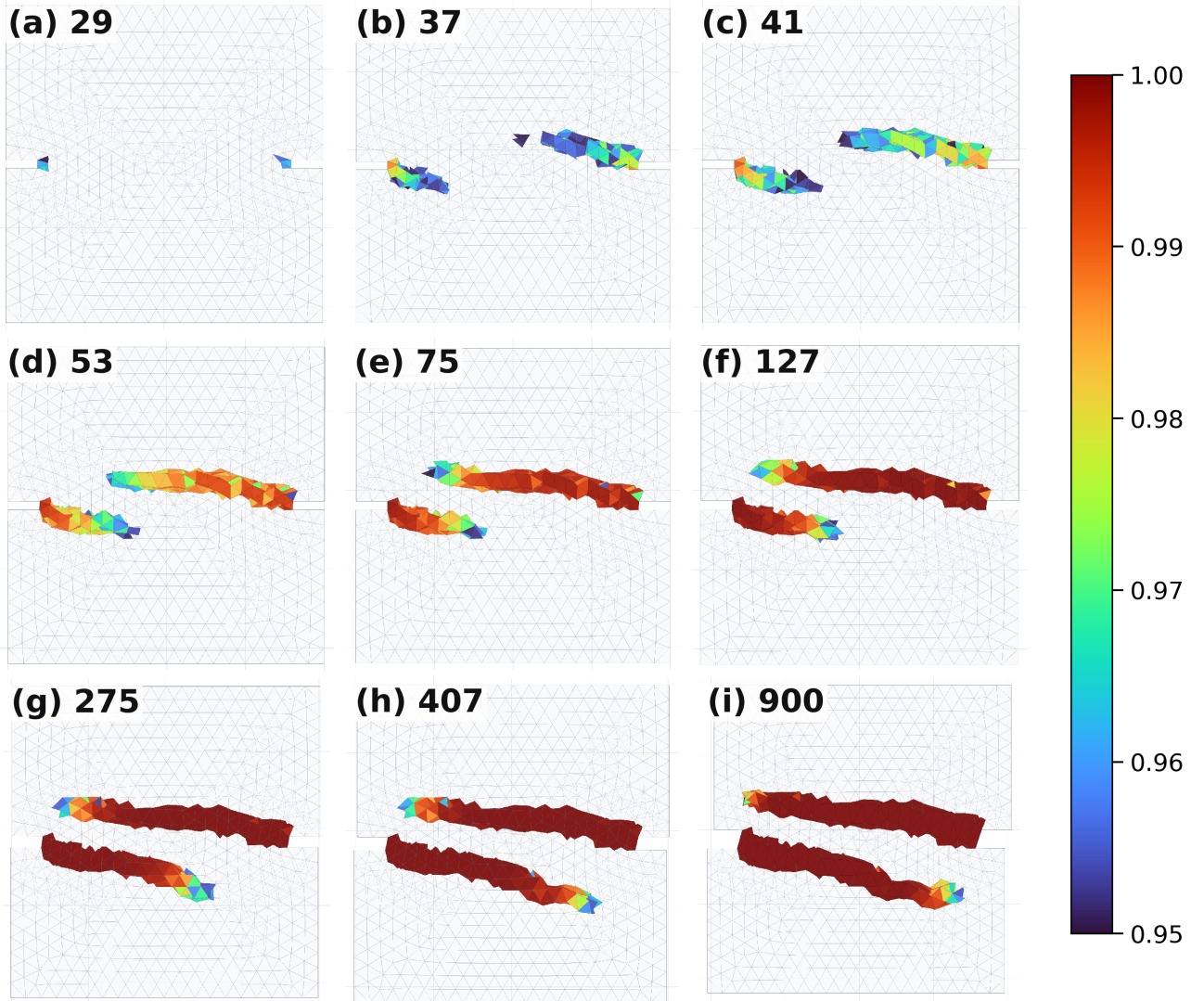
The completed runs are presented as response comparisons for this mesh family. Partial dissipation is evaluated at the same CMOD in every run.



(a) 2D boundary conditions.

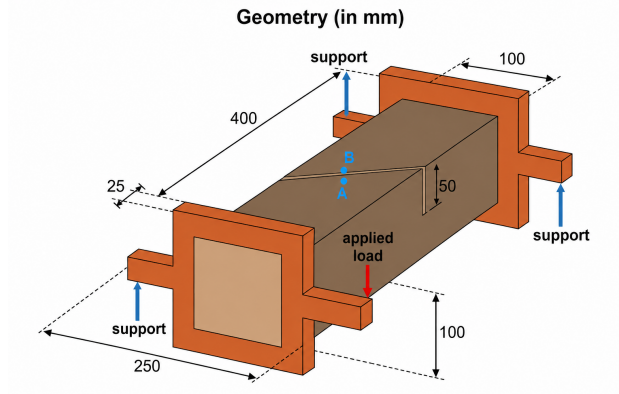


(b) Pure-tension simulations on three meshes; 25 mm notches.

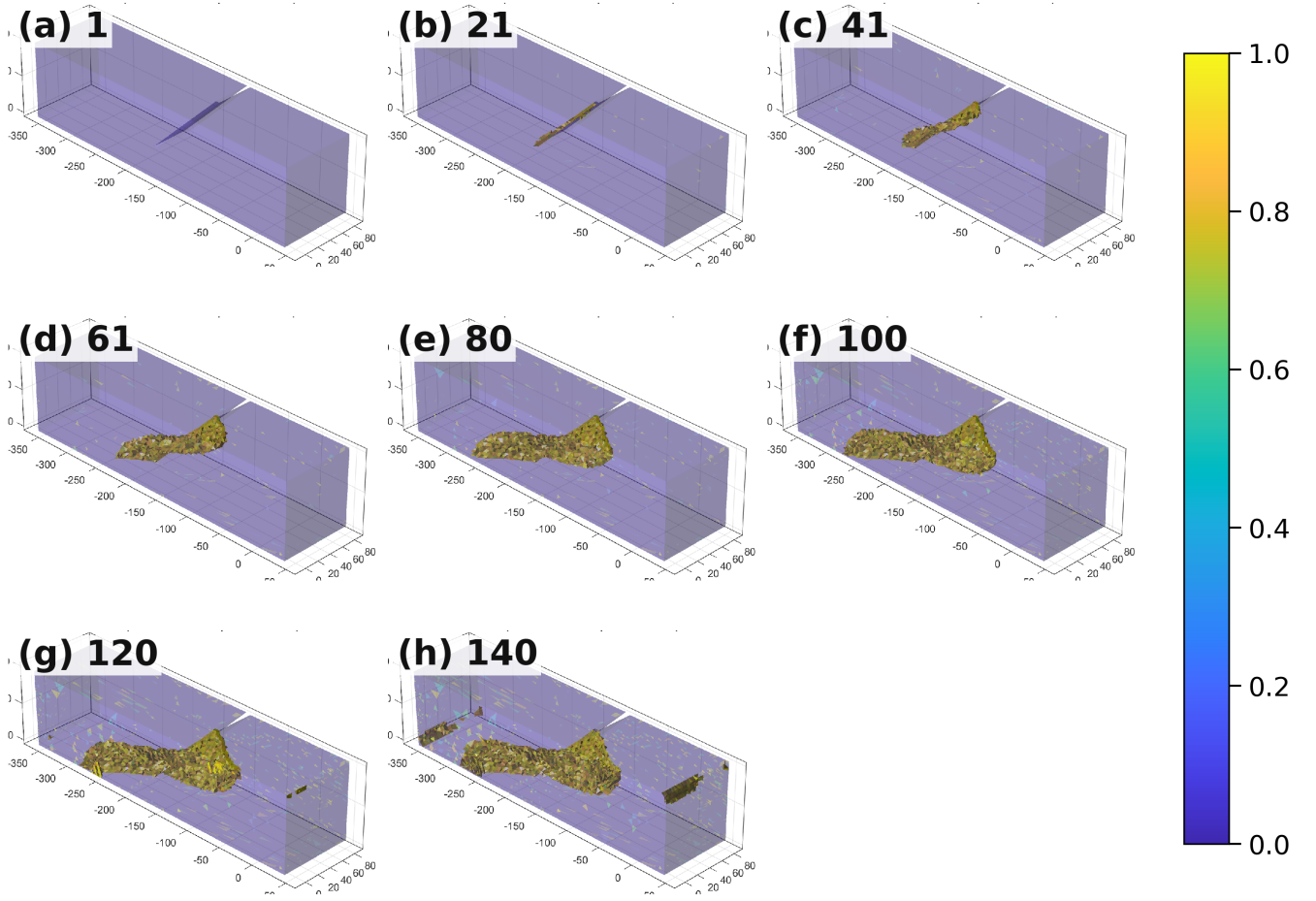


(c) Damage evolution from first localization at the notches to coalescence; panels (a)–(i), left to right and top to bottom, correspond to increments 29, 37, 41, 53, 75, 127, 275, 407, and 900.

Figure 3: Nooru-Mohamed cases: proportional-loading boundary conditions and damage panels; the load curves compare three separate pure-tension meshes. Only simulation curves are shown. Proportional-loading panels use 20 mm notches; the pure-tension curves use 25 mm notches. The controls differ. Mixed-mode damage panels are qualitative; their recorded reaction history contains equilibrium failures.



(a) Published collar loading arrangement; the illustrative model imposes an end-face twist.



(b) Torsion damage sequence.

Figure 4: Brokenshire torsion example. The 3D panels illustrate the workflow qualitatively; quantitative validation against a measured torque curve is outside this study.

Table 5: Controlled mesh study, 2,000 MATLAB increments. Partial damage dissipation $\mathcal{D}$ is evaluated at CMOD 0.10 mm. Subscripts O, F, and A denote Oliver MATLAB, constant-width MATLAB, and regularized Abaqus runs. The dissipation columns quantify the effect of element-size compensation beyond the load–CMOD curves.

Elements	DOFs	P_O (kN)	P_F (kN)	P_A (kN)	$\mathcal{D}_O$ (N mm)	$\mathcal{D}_F$ (N mm)
14,313	14,686	4.194	4.025	4.117	140.08	132.13
25,042	25,534	4.205	3.808	4.087	140.93	117.25
56,216	56,954	4.459	3.763	4.339	149.94	110.30

MATLAB peaks exceed Abaqus by 1.87%, 2.88% and 2.78%. The Abaqus jobs have 2/0/0 cutbacks, 5,152/6,153/7,502 decompositions and 4/3/5 alternate force-tolerance acceptances. Their reported CPU totals are 531/1,100/2,550 s; wall time outside sparse-solver scopes is 794.2/1,585.6/3,979.7 s. These different timer scopes do not isolate material or assembly work. Exact meshes, input decks, complete curves, diagnostics and analysis scripts accompany the study.

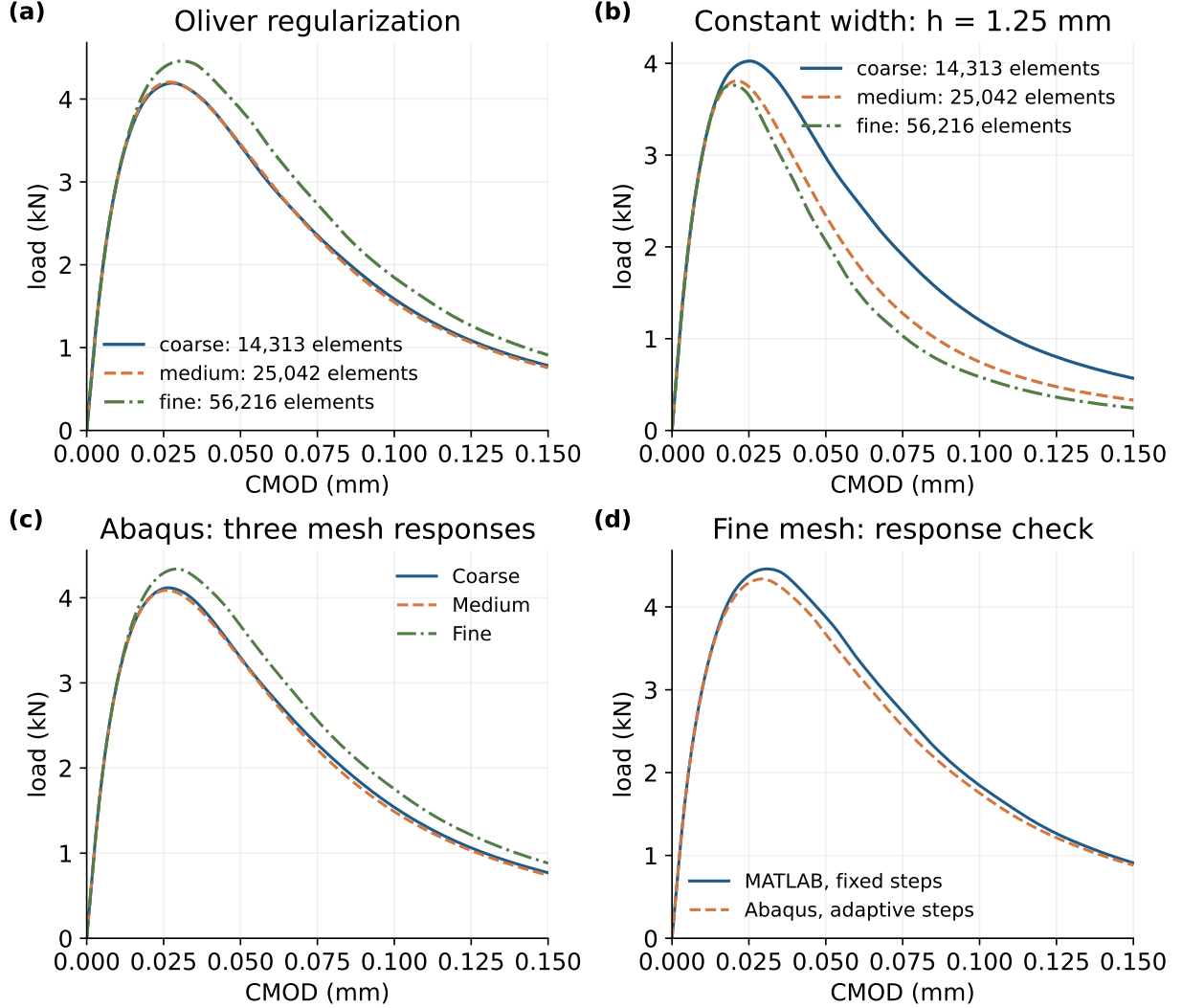


Figure 5: Completed 2D responses on three meshes. (a) Oliver regularization; (b) constant-width control using the same exponential softening curve with $h = 1.25$ mm on every mesh; (c) regularized Abaqus responses; (d) fine-mesh MATLAB/Abaqus comparison. MATLAB curves use 2,000 increments to -0.1 mm. Abaqus uses the same maximum increment with adaptive cutbacks.

4. Impact

FRACMATH makes implementation choices observable through source code, numerical histories and diagnostic scripts. It supports research into how projected crack-band width, irreversible damage and increment size affect structural response. The constant-width control separates element-size compensation from the remaining numerical sensitivity. In the controlled 2D family, regularization reduces partial-dissipation spread from 18.21% to 6.86%; the smaller-increment family retains this reduction. Residual and energy records let users judge the associated equilibrium error before extending the method.

The 3D tension example shows how to distinguish mesh sensitivity from experimental agreement: a 1.46% peak spread coexists with 16–17% experimental underprediction. Its interpolated gauge

control, rejected-trial handling and digitized reference provide inspectable starting points for studying boundary conditions and constitutive assumptions.

The timing results show which parts of the calculation take most time. Ninety observations on matched meshes show that moving element operations to a GPU does not improve this implementation on the tested workstation, where CPU sparse factorization dominates. Component scopes and Abaqus iteration counts allow subsequent implementations to be compared with explicit accuracy and timing definitions.

For teaching and reuse, the entry script and array guide connect each input to a computed state and plotted response. The verified UMAT provides a second implementation for examining strain conventions and regularization. Users can reproduce the figures, change a documented routine and inspect the resulting response, energy and residual histories. Independent adoption, download statistics and commercial use have not been established.

4.1. Limitations

The model is local, isotropic and quasistatic, with one damage scalar; crack closure and anisotropic degradation are unrepresented. The reported mesh families do not establish general mesh or orientation independence. The 3D tension peaks remain 16.18–17.39% below experiment; mixed-mode and torsion fields remain qualitative. The 2D sequential update leaves a post-damage equilibrium residual; coupled outer iteration is future work. The hybrid GPU supports 2D T3 elements with CPU sparse factorization. Three timing observations per configuration on one workstation provide observed ranges. Different equilibrium algorithms and timer scopes prevent an accuracy-matched cross-program speed ranking.

5. Conclusions

FRACMATH is an inspectable MATLAB example of local damage with an Oliver projected crack-band width. The study presents constitutive checks, controlled 2D mesh and increment studies, structural energy diagnostics, and timing scopes that separate sparse factorization from assembly in MATLAB. A geometry-matched 3D pure-tension comparison quantifies peak and curve errors without fitting material parameters. The hybrid GPU and SMP study reports 90 timing observations on matched meshes. The mixed-mode and torsion cases remain qualitative, and direct timers record UMAT call duration with explicit clock diagnostics. Native profiling identifies assembly self work; complete assembly wall cost remains unallocated.

Data availability

The MATLAB and UMAT sources, 2D/3D meshes, material-point tests, machine-readable response histories, timing observations and figure scripts are available in the [FRACMATH GitHub repository](#) [12]. The core archive and the timing and figure companions are identified by separate immutable Git commits listed in Table 1.

CRedit authorship contribution statement

Jaykumar Mavani: Methodology, Software, Validation, Visualization, Writing – original draft.
Madura Pathirage: Conceptualization, Supervision, Funding acquisition.

Acknowledgements

The authors acknowledge support from the National Aeronautics and Space Administration (NASA), and the New Mexico Space Grant Consortium under grant 80NSSC22M0044. The funders had no role in study design, data analysis, manuscript writing or the decision to submit for publication.

Declaration of competing interest

The authors declare no conflict of interest.

Declaration of AI-assisted technologies

Generative AI tools were used to assist with manuscript wording, formatting, and implementation support. The scientific claims, equations, code, numerical results, figures, and references are the responsibility of the authors.

References

- [1] J. H. P. de Vree, W. A. M. Brekelmans, M. A. J. van Gils, Comparison of nonlocal approaches in continuum damage mechanics, *Computers & Structures* 55 (4) (1995) 581–588. doi:10.1016/0045-7949(94)00501-S.
- [2] J. Oliver, A consistent characteristic length for smeared cracking models, *International Journal for Numerical Methods in Engineering* 28 (2) (1989) 461–474. doi:10.1002/nme.1620280214.
- [3] Z. P. Bažant, B. H. Oh, Crack band theory for fracture of concrete, *Matériaux et Construction* 16 (3) (1983) 155–177. doi:10.1007/BF02486267.
- [4] Dassault Systèmes Simulia Corp., *Abaqus/Standard User's Manual*, Version 2024, Providence, RI, USA, 2024.
- [5] B. Patzák, Z. Bittnar, Design of object-oriented finite element code, *Advances in Engineering Software* 32 (10–11) (2001) 759–767. doi:10.1016/S0965-9978(01)00027-8.
- [6] P.-E. Austrell, O. Dahlblom, J. Lindemann, A. Olsson, K.-G. Olsson, K. Persson, H. Petersson, M. Ristinmaa, G. Sandberg, P.-A. Wernberg, *CALFEM—A Finite Element Toolbox*, Version 3.4, Lund University, Lund, Sweden, 2004.
- [7] D. Grégoire, L. B. Rojas-Solano, G. Pijaudier-Cabot, Failure and size effect for notched and unnotched concrete beams, *International Journal for Numerical and Analytical Methods in Geomechanics* 37 (10) (2013) 1434–1452. doi:10.1002/nag.2180.
- [8] M. B. Nooru-Mohamed, Mixed-mode fracture of concrete: an experimental approach, Ph.D. thesis, Delft University of Technology, 1992. URL: <https://resolver.tudelft.nl/uuid:a6a773f1-dacd-4598-aa6a-960dddf71117>.
- [9] D. R. Brokenshire, A study of torsion fracture tests, Ph.D. thesis, Cardiff University, Cardiff, United Kingdom, 1996.
- [10] A. D. Jefferson, B. I. G. Barr, T. Bennett, S. C. Hee, Three dimensional finite element simulations of fracture tests using the Craft concrete model, *Computers and Concrete* 1 (3) (2004) 261–284. doi:10.12989/cac.2004.1.3.261.
- [11] A. D. Jefferson, T. Bennett, Benchmark problems for assessing a combined micro-mechanical continuum based model for concrete, COMPLAS VIII, Barcelona, 2005. URL: <https://congress.cimne.com/complas05/admin/Files/FilePaper/p334.pdf>.
- [12] J. Mavani, M. Pathirage, FRACMATH, Version 1.1.1, Zenodo, 2026. doi:10.5281/zenodo.23138595. Git commit f4d208e. URL: <https://github.com/Jaykumar9033/FRACMATH/tree/f4d208ecf45b6a4e4d4530e5f71fbf8bcb9f0fe0>.